

# 抽取 Wednesday 流量与数据包外部测试集

会话 ID: 019fad06-3107-7fb2-a811-d5e06fc9dee3 开始时间: 2026-07-29 10:38:12 消息数: 13

用户 2026-07-29 10:38:14

我的dataset下面Wednesday-workingHours-stage1\_flows.csv和Wednesday-workingHours-stage1\_packets.csv外部的数据，太多了，我想抽取一部分，按照时间抽取flow，然后从packets中抽取相关的flow对应的packet。我想生成多个小的文件，用于外部测试集进行验证。结合我现有的代码，我该如何做？

助手 2026-07-29 10:38:19

我先检查现有的数据加载、flow标识和时间字段处理逻辑，再给你一个能直接生成多组外部测试集脚本，并确保输出格式能被当前代码读取。

助手 2026-07-29 10:38:53

项目里已经有两个相关脚本。我正在核对它们是否正好满足“按时间分成多组flow，并用flow\_id流式筛选packets”，避免把超大的packets CSV一次性载入内存。

助手 2026-07-29 10:39:25

数据规模约为flow 248 MB、packets 2.73 GB。最合适的实现是：先把flow按开始时间排序并切成互不重叠的小窗口；再只扫描packets一遍，同时把每个flow\_id的包写到对应窗口。这样生成5组也不会把2.73 GB文件重复读5次。

助手 2026-07-29 10:40:47

已经结合现有Stage1代码新增了脚本：`[make_multiple_time_external_tests.py](C:\Users\XiaoyanXiong\Desktop\NIDSTransformer\s1\make_multiple_time_external_tests.py)`。

它会：

- 按`flow\_start\_timestamp\_us`排序flow。
- 生成多个互不重叠的时间窗口。
- 根据`flow\_id`提取对应packets。
- 对2.73 GB packets文件只扫描一次。
- 每组生成配套的`flows.csv`和`packets.csv`。
- 输出JSON报告，记录时间范围、类别数量、缺失packet的flow数量等。

例如生成5组，每组5000个flow：

```
python s1/make_multiple_time_external_tests.py `
--flow_csv dataset/Wednesday-workingHours-stage1_flows.csv `
--packet_csv dataset/Wednesday-workingHours-stage1_packets.csv `
--out_dir dataset/wednesday_external_windows `
--n_windows 5 `
--flows_per_window 5000 `
--placement even
```

`--placement even`会把5个窗口均匀分布在整个Wednesday时间线上，更适合验证不同时段模型表现。

如果想连续截取5段：

```
python s1/make_multiple_time_external_tests.py `
```

```
--n_windows 5 `
--flows_per_window 5000 `
--placement consecutive `
--start_fraction 0.5
```

这表示从整个时间线约 50% 的位置开始，连续切出 5 组。

输出结构类似：

```
dataset/wednesday_external_windows/
├── Wednesday-workingHours-external-w01-stage1_flows.csv
├── Wednesday-workingHours-external-w01-stage1_packets.csv
├── Wednesday-workingHours-external-w02-stage1_flows.csv
├── Wednesday-workingHours-external-w02-stage1_packets.csv
├── ...
└── Wednesday-workingHours-external-windows-report.json
```

然后把其中一组传给现有 Stage1：

```
python s1/run_stage1.py `
--packet_csv < _packets.csv> `
--flow_csv < _flows.csv> `
--external_packet_csv dataset/wednesday_external_windows/Wednesday-workingHours-external-w01-stage1_packets.csv `
--external_flow_csv dataset/wednesday_external_windows/Wednesday-workingHours-external-w01-stage1_flows.csv `
--config < .yaml> `
--out_dir s1/results/external_w01
```

需要分别验证 5 组时，只需依次将 `w01` 换成 `w02` 到 `w05`。

脚本已通过语法和命令参数检查。由于 packets 原文件有约 2.73

GB，我没有直接运行完整抽取；运行时建议保留至少数 GB 可用磁盘空间。生成后重点查看报告中的：

- `benign\_count`
- `malicious\_count`
- `matched\_flow\_count`
- `missing\_packet\_flow\_count`

如果某个时间窗口的 `malicious\_count=0`，该窗口只能用于测试正常流误报率，不适合单独计算攻击召回率。

用户 2026-07-29 10:56:08

```
df = pd.read_csv("dataset/Wednesday-workingHours-stage1_flows.csv")
df = df[(df['flow_id'] != 0) & (df['flow_start_timestamp_us'] != 0) &
df['flow_id'].notna() & df['flow_start_timestamp_us'].notna()]
```

```
print("df.shape", df.shape)
```

```
df.shape (475809, 78)
```

这是数据的信息，结合我的数据信息我该如何生成外部数据？

助手 2026-07-29 10:56:48

你的 flow 数据清洗后有 `475,809` 条，因此很适合按时间建立多个互不重叠的外部测试窗口。

建议先生成：

- 5 个外部测试集
- 每组 5,000 个 flow
- 共使用 25,000 个 flow，约占全部数据的 5.25%
- 每组 packets 都按照 `flow\_id` 精确匹配

- 5 组分布在整个时间线上

运行：

```
python s1/make_multiple_time_external_tests.py `
--flow_csv dataset/Wednesday-workingHours-stage1_flows.csv `
--packet_csv dataset/Wednesday-workingHours-stage1_packets.csv `
--out_dir dataset/wednesday_external_windows `
--n_windows 5 `
--flows_per_window 5000 `
--placement even
```

对于你的 `475,809` 条 flow，这 5 个窗口大致位于：

```
| 测试集 | 排序后 flow 行范围 | 数据作用 |
|---|---|---|
| w01 | 0-4,999 | 最早时段 |
| w02 | 117,702-122,701 | 前中段 |
| w03 | 235,405-240,404 | 中间时段 |
| w04 | 353,107-358,106 | 后中段 |
| w05 | 470,809-475,808 | 最晚时段 |
```

这里的排序依据是：

```
df.sort_values("flow_start_timestamp_us")
```

输出结果为：

```
dataset/wednesday_external_windows/
Wednesday-workingHours-external-w01-stage1_flows.csv
Wednesday-workingHours-external-w01-stage1_packets.csv
...
Wednesday-workingHours-external-w05-stage1_flows.csv
Wednesday-workingHours-external-w05-stage1_packets.csv
Wednesday-workingHours-external-windows-report.json
```

生成后检查类别分布

Wednesday 数据中的攻击通常不是均匀分布在全天时间线上的。因此，均匀抽取的某些窗口可能全部是 benign。先运行：

```
import json

report_path = (
    "dataset/wednesday_external_windows/"
    "Wednesday-workingHours-external-windows-report.json"
)

with open(report_path, encoding="utf-8") as f:
    report = json.load(f)

for w in report["windows"]:
    print(
        w["name"],
        "flows =", w["flow_count"],
        "benign =", w["benign_count"],
        "malicious =", w["malicious_count"],
        "packets =", w["packet_count"],
        "missing =", w["missing_packet_flow_count"],
    )
```

理想情况下：

```
malicious_count > 0
missing_packet_flow_count == 0
```

```
packet_count > 0
```

如果某个窗口的 `malicious\_count == 0`，它仍然可以用于评估误报率，但不能单独用于评估攻击召回率。

先查看攻击所在时间段

可以用下面代码确定攻击 flow 在时间线上的分布：

```
import pandas as pd

df = pd.read_csv(
    "dataset/Wednesday-workingHours-stage1_flows.csv",
    usecols=["flow_id", "flow_start_timestamp_us", "label"],
)

df = df[
    (df["flow_id"] != 0)
    & (df["flow_start_timestamp_us"] != 0)
    & df["flow_id"].notna()
    & df["flow_start_timestamp_us"].notna()
].copy()

df = df.sort_values("flow_start_timestamp_us").reset_index(drop=True)
df["time"] = pd.to_datetime(df["flow_start_timestamp_us"], unit="us")

print(" flow ", len(df))
print("\n ")
print(df["label"].value_counts())

print("\n ")
print(
    df.groupby([df["time"].dt.floor("h"), "label"])
    .size()
    .unstack(fill_value=0)
)
```

如果 `label` 是 `0/1`，还可以输出每 10,000 条 flow 的类别分布：

```
df["time_block"] = df.index // 10_000

distribution = (
    df.groupby(["time_block", "label"])
    .size()
    .unstack(fill_value=0)
)

print(distribution.to_string())
```

这样可以找到同时含 benign 和 malicious 的时间段。

如何用于现有 Stage1

例如测试第一组：

```
python s1/run_stage1.py `
--packet_csv dataset/train-stage1_packets.csv `
--flow_csv dataset/train-stage1_flows.csv `
--external_packet_csv dataset/wednesday_external_windows/Wednesday-workingHours-external-w01-stage1_packets.csv `
--external_flow_csv dataset/wednesday_external_windows/Wednesday-workingHours-external-w01-stage1_flows.csv `
--config s1/stage1_config_0626_B_Both.yaml `
--out_dir s1/results/wednesday_external_w01
```

之后依次测试 `w02` ~ `w05`。

需要特别注意数据泄漏

如果你的训练集也来自这份完整的 Wednesday 数据，那么这些外部测试窗口对应的 flow 必须从训练数据中删除。否则同一个 `flow\_id` 同时出现在训练集和外部测试集，结果会虚高。

可以检查：

```
train_ids = set(train_flows["flow_id"])
external_ids = set(external_flows["flow_id"])
```

```
overlap = train_ids & external_ids
print("          flow      ", len(overlap))
```

必须得到：

```
flow      0
```

如果 Wednesday 完全没有参与模型训练，它才是真正意义上的独立外部测试集；如果从 Wednesday 内部切出测试窗口，更准确的名称是“基于时间切分的留出测试集”。

## 用户 2026-07-29 11:19:35

make\_multiple\_time\_external\_tests.py

详细解析是如何获取数据的？是按时间连续取的吗？还是随机取得？详细解析

## 助手 2026-07-29 11:20:59

`make\_multiple\_time\_external\_tests.py` 不是随机抽样。它会先按时间排序，然后为每个测试集截取一段连续的 flow。

不过，“多个窗口之间是否连续”取决于 `--placement`：

- `even`：每个窗口内部连续，但多个窗口分散在整个时间轴上。
- `consecutive`：每个窗口内部连续，而且多个窗口彼此相邻。

### ## 1. 总体处理流程

脚本按照下面的顺序获取数据：

```
flow CSV
↓
flow
↓
flow_start_timestamp_us
↓
↓
↓
flow CSV
↓
flow_id →
↓
packets CSV
↓
flow_id packet
```

所以 packets 不是按 packet 时间单独抽取，而是根据已经选中的 `flow\_id` 进行关联提取。

---

### ## 2. Flow 数据如何清洗

脚本首先读取：

```
flows = pd.read_csv(path, low_memory=False)
```

然后检查必须存在以下三列：

```
required = {
    "flow_id",
    "flow_start_timestamp_us",
    "label",
}
```

接着把 `flow\_id` 和时间转换为数值：

```
flows["flow_id"] = pd.to_numeric(
    flows["flow_id"],
    errors="coerce",
)

flows["flow_start_timestamp_us"] = pd.to_numeric(
    flows["flow_start_timestamp_us"],
    errors="coerce",
)
```

无法转换的值会变成 `NaN`。

清洗条件和你现在使用的逻辑一致：

```
flows = flows[
    flows["flow_id"].notna()
    & flows["flow_start_timestamp_us"].notna()
    & (flows["flow_id"] != 0)
    & (flows["flow_start_timestamp_us"] != 0)
].copy()
```

也就是删除：

- `flow\_id == 0`
- `flow\_id` 为空
- `flow\_start\_timestamp\_us == 0`
- `flow\_start\_timestamp\_us` 为空

你的数据清洗后是：

```
475,809 flows
```

```
---
```

### ## 3. 是否删除重复 flow\_id

脚本会执行：

```
flows = flows.drop_duplicates(
    "flow_id",
    keep="first",
)
```

如果一个 `flow\_id` 在 flow CSV 中重复出现，只保留第一次出现的记录。

这是为了保证：

- 一个 flow 只能属于一个外部测试窗口。
- 同一个 `flow\_id` 不会被重复验证。
- packets 可以明确地对应到一个测试窗口。

这里的“第一次”指原始 CSV 中首次出现的记录，因为去重发生在时间排序之前。

```
---
```

### ## 4. 如何按时间排序

清洗、去重后，执行：

```
flows = flows.sort_values(
    ["flow_start_timestamp_us", "flow_id"],
    kind="mergesort",
).reset_index(drop=True)
```

主要排序字段是：

```
flow_start_timestamp_us
```

表示按照 flow 的开始时间从早到晚排列。

如果两个 flow 的开始时间相同，再按照：

```
flow_id
```

排序，确保每次运行得到相同顺序。

因此，排序后的数据可以理解为：

```
0          flow
1          flow
...
475808      flow
```

这一步没有随机性。

---

# `--placement even` 的行为

假设运行：

```
python s1/make_multiple_time_external_tests.py `
--n_windows 5 `
--flows_per_window 5000 `
--placement even
```

含义是：

- 创建 5 个窗口。
- 每个窗口包含 5,000 个 flow。
- 窗口均匀分布在完整时间线上。

## 5. 如何计算窗口起点

你的总 flow 数为：

```
total = 475809
```

每个窗口大小：

```
size = 5000
```

一个完整窗口最晚可以从以下位置开始：

```
max_start = total - size
```

即：

```
475809 - 5000 = 470809
```

5 个窗口的起点使用下面公式：

```
start_i = round(
    base
    + i * (remaining_max_start - base)
    / (count - 1)
```

)

默认:

```
base = 0
count = 5
remaining_max_start = 470809
```

所以起点大约是:

```
w01: 0
w02: 117702
w03: 235404    235405
w04: 353107
w05: 470809
```

每组通过 `iloc` 连续截取:

```
selected = flows.iloc[
    start : start + flows_per_window
].copy()
```

对应范围大约为:

| 窗口  | 起始索引    | 结束索引    | 是否连续   |
|-----|---------|---------|--------|
| w01 | 0       | 4,999   | 窗口内部连续 |
| w02 | 117,702 | 122,701 | 窗口内部连续 |
| w03 | 235,404 | 240,403 | 窗口内部连续 |
| w04 | 353,107 | 358,106 | 窗口内部连续 |
| w05 | 470,809 | 475,808 | 窗口内部连续 |

注意: 这些是排序后的行号范围, 不一定代表固定的时间长度。

例如:

- 某 5,000 个 flow 可能覆盖 5 分钟。
- 另一个 5,000 个 flow 可能覆盖 30 分钟。

这是因为脚本固定的是 “每个窗口的 flow 数量”, 不是 “每个窗口覆盖多少分钟”。

因此, `even` 模式的准确描述是:

> 在完整时间轴的不同位置, 均匀选择多个窗口; 每个窗口内部包含按开始时间连续排列的 5,000 个 flow。

它不是:

- 随机选择 5,000 个 flow。
- 每隔固定数量选择一个 flow。
- 固定每小时抽取一部分。
- 随机打乱后抽取。

---

# `--placement consecutive` 的行为

例如:

```
python s1/make_multiple_time_external_tests.py `
--n_windows 5 `
--flows_per_window 5000 `
--placement consecutive `
--start_fraction 0.5
```



`start\_fraction=0.5` 表示从排序后大约 50% 的位置开始：

```
base = int(total * start_fraction)
```

你的数据大约是：

```
base = int(475809 * 0.5)
      = 237904
```

窗口起点使用：

```
starts = [
    base + i * size
    for i in range(count)
]
```

因此大致为：

```
| 窗口 | 排序后行范围 |
|---|---|
| w01 | 237,904–242,903 |
| w02 | 242,904–247,903 |
| w03 | 247,904–252,903 |
| w04 | 252,904–257,903 |
| w05 | 257,904–262,903 |
```

这种情况下：

```
w01 → w02 → w03 → w04 → w05
```

完全相邻，没有间隔，也没有重叠。

注意，边界上的 flow 仍然可能在时间上重叠。例如：

- w01 最后一条 flow 开始于 10:00:05。
- w02 第一条 flow 也可能开始于 10:00:05。

因为划分依据是 `flow\_start\_timestamp\_us` 的排序行号，不是严格互斥的墙钟时间区间。但同一个 `flow\_id` 不会重复。

---

## ## 6. `start\_fraction` 的意义

它表示从完整时间线的哪个相对位置开始。

例如：

```
--start_fraction 0.0
```

从最早位置开始。

```
--start_fraction 0.25
```

从大约 25% 的位置开始。

```
--start_fraction 0.5
```

从中间位置开始。

```
--start_fraction 0.8
```

从大约 80% 的位置开始。

它是根据排序后的 flow 行数计算，不是直接根据时间戳范围计算。因此 `0.5` 表示 “第 50% 个 flow”，不一定表示当天时间跨度的正中间。

---

# Packets 是如何提取的

## 7. 先建立 flow\_id 映射

每个窗口的 flow 选好之后，脚本建立：

```
flow_to_window: dict[int, int] = {}
```

例如：

```
flow_id 10001 → w01
flow_id 10002 → w01
flow_id 20001 → w02
flow_id 20002 → w02
```

对应代码逻辑：

```
for flow_id in ids:
    flow_to_window[flow_id] = i - 1
```

同时检查一个 `flow\_id` 是否进入多个窗口：

```
if flow_id in flow_to_window:
    raise RuntimeError(...)
```

所以不同输出窗口之间不会有重复 flow。

---

## 8. packets CSV 不会一次性读入内存

完整 packets 文件约 2.73 GB。脚本使用：

```
pd.read_csv(
    packet_path,
    chunksize=args.chunksize,
    low_memory=False,
)
```

默认：

```
chunksize = 500000
```

也就是一次读取 50 万条 packet。

假设 packets 一共有很多百万行，处理方式类似：

```
1 50
```

```
2 50
```

```
...
```

```
packets CSV
```

这样不会把整个 2.73 GB 文件放进内存。

---

## 9. 每条 packet 如何匹配窗口

首先提取 packet 的 `flow\_id`:

```
ids = pd.to_numeric(
    chunk["flow_id"],
    errors="coerce",
).astype("Int64")
```

然后通过映射寻找目标窗口:

```
destinations = ids.map(flow_to_window)
```

结果可能类似:

```
| packet flow_id | destination |
|---:|---:|
| 10001 | 0, 即 w01 |
| 10002 | 0, 即 w01 |
| 20001 | 1, 即 w02 |
| 99999 | NaN, 不保留 |
```

如果 packet 的 `flow\_id` 不属于任何选中的 flow, 该 packet 会被丢弃。

如果属于某个窗口, 就写入对应文件:

```
sub.to_csv(
    packet_out,
    mode="a",
)
```

因此, 对于每个外部测试窗口:

```
window flow CSV      flow_id
      ↓
packets CSV      flow_id  packet
```

---

# Packet 是否也按连续时间截取

不是。

脚本不会使用下面这种条件直接切 packets:

```
packets[
    (packets["timestamp_us"] >= start_time)
    & (packets["timestamp_us"] <= end_time)
]
```

它使用的是:

```
packets["flow_id"].isin(selected_flow_ids)
```

这是更适合你当前 Stage1 数据结构的方法。

原因是一个 flow 可能:

- 在窗口结束位置之前开始;
- 但最后几个 packet 出现在窗口边界之后。

如果直接按 packet 时间截取, 可能只保留一个 flow 的部分 packets, 导致 flow 序列不完整。

当前脚本根据 `flow\_id` 保留相关的所有 packet, 因此:

> 只要一个 flow 被选中, 它在 packets CSV 中对应的所有 packet 都会被保留。

---

# 输出文件中的顺序

## Flow 文件

Flow 文件按照：

```
flow_start_timestamp_us  
flow_id
```

排序。

所以 flow 输出是时间有序的。

## Packet 文件

Packet 文件保留其在原始 packets CSV 中的出现顺序。

脚本没有对输出 packet 再进行排序，因为：

- 它按照原 CSV 的块顺序读取；
- 块内通过布尔条件筛选，但不打乱行；
- 后续块追加到文件末尾。

如果原始 packets CSV 本身按时间或捕获顺序排列，输出也会保持该顺序。

Stage1 内部还会按以下字段组织每个 flow 的 packet：

```
flow_id  
timestamp_us  
packet_id
```

因此输出文件不需要先把所有 packet 放入内存重新排序。

---

# 标签如何处理

脚本不会随机平衡 benign 和 malicious，也不会主动保证每个窗口都有攻击。

它只是读取选中 flow 的 `label`，统计：

```
labels = pd.to_numeric(  
    selected["label"],  
    errors="coerce",  
).fillna(0)  
  
binary = (labels != 0).astype(int)
```

也就是说：

```
label == 0 → benign  
label != 0 → malicious
```

然后报告：

```
"benign_count"  
"malicious_count"
```

因此，`even` 仅保证窗口均匀分散，不保证类别均衡。

这是时间外部验证与随机分层抽样的主要区别：

|                  |      |      |      |
|------------------|------|------|------|
| 方法               | 时间结构 | 类别保证 | 是否随机 |
| ---              | ---  | ---  | ---  |
| 当前 `even`        | 保留   | 不保证  | 否    |
| 当前 `consecutive` | 完全保留 | 不保证  | 否    |

| 随机分层抽样 | 破坏 | 可以保证 | 是 |  
| 攻击感知时间窗口 | 保留 | 可以设置最低攻击数 | 否 |

---

# 最终结论

默认命令:

```
python s1/make_multiple_time_external_tests.py `
--n_windows 5 `
--flows_per_window 5000 `
--placement even
```

实际执行的是:

1. 清洗 flow。
2. 删除重复 `flow\_id`。
3. 按 `flow\_start\_timestamp\_us` 从早到晚排序。
4. 在整个时间线上均匀确定 5 个窗口起点。
5. 从每个起点连续取 5,000 个 flow。
6. 不随机、不打乱、不分层采样。
7. 根据选中 flow 的 `flow\_id`, 从 packets 中提取全部相关 packet。
8. packets 原文件只扫描一次。
9. 不属于选中 flow 的 packet 全部丢弃。

所以最准确的描述是:

> 它采用确定性的、按 flow 开始时间排序的连续窗口抽取。`even` 模式下每个窗口内部连续, 但窗口之间分散; `consecutive` 模式下所有窗口彼此连续。整个过程没有随机抽样。

用户 2026-07-29 12:09:54

如何验证取出来的数据是按照时间顺序取出来的?

助手 2026-07-29 12:10:45

可以从三个层面验证:

1. 每个 flow 文件内部是否按时间递增。
2. 抽出的 flow 是否确实是原始数据排序后的连续行。
3. 多个窗口之间是否符合 `even` 或 `consecutive` 的位置关系。

最可靠的是第 2 种, 因为“时间递增”只能证明排过序, 不能证明中间没有跳着抽取。

## 一、快速检查每个文件是否时间有序

```
import pandas as pd
from pathlib import Path

out_dir = Path("dataset/wednesday_external_windows")

flow_files = sorted(out_dir.glob("*-stage1_flows.csv"))

for path in flow_files:
    df = pd.read_csv(
        path,
        usecols=["flow_id", "flow_start_timestamp_us", "label"],
    )
```

```

timestamps = pd.to_numeric(
    df["flow_start_timestamp_us"],
    errors="coerce",
)

is_sorted = timestamps.is_monotonic_increasing

print(
    path.name,
    "flow    =", len(df),
    "    =", is_sorted,
    "    =", timestamps.iloc[0],
    "    =", timestamps.iloc[-1],
)

```

如果正确，应该看到：

```
= True
```

`is\_monotonic\_increasing` 允许多个 flow 具有相同开始时间。

如果要求时间严格递增，可以检查：

```

is_strictly_increasing = (
    timestamps.diff().dropna() > 0
).all()

```

但实际网络数据中多个 flow 可能同时开始，因此严格递增为 `False` 不一定是错误。

---

## ## 二、找出是否存在时间逆序

如果某个文件的结果是：

```
= False
```

可以定位具体异常位置：

```

import pandas as pd

path = (
    "dataset/wednesday_external_windows/"
    "Wednesday-workingHours-external-w01-stage1_flows.csv"
)

df = pd.read_csv(path)

ts = pd.to_numeric(
    df["flow_start_timestamp_us"],
    errors="coerce",
)

bad = ts.diff() < 0

print("    ", bad.sum())

if bad.any():
    bad_positions = bad[bad].index

    for pos in bad_positions[:10]:
        print(df.loc[
            max(0, pos - 1):pos,
            ["flow_id", "flow_start_timestamp_us", "label"],
        ])

```

正确结果应该是：

```
0
```

---

### ## 三、验证它是原始数据中的连续切片

这是最重要的检查。

脚本生成的报告中记录了每个窗口的：

```
start_row
flow_count
```

因此可以重新清洗、排序原始 flow 数据，然后检查输出文件是否和原始排序结果的对应切片完全一致。

创建检查代码：

```
import json
from pathlib import Path

import pandas as pd

source_path = Path(
    "dataset/Wednesday-workingHours-stage1_flows.csv"
)

report_path = Path(
    "dataset/wednesday_external_windows/"
    "Wednesday-workingHours-external-windows-report.json"
)

# 1.          flow
source = pd.read_csv(source_path, low_memory=False)

source.columns = [
    str(column).strip()
    for column in source.columns
]

source["flow_id"] = pd.to_numeric(
    source["flow_id"],
    errors="coerce",
)

source["flow_start_timestamp_us"] = pd.to_numeric(
    source["flow_start_timestamp_us"],
    errors="coerce",
)

# 2.
source = source[
    source["flow_id"].notna()
    & source["flow_start_timestamp_us"].notna()
    & (source["flow_id"] != 0)
    & (source["flow_start_timestamp_us"] != 0)
].copy()

source["flow_id"] = source["flow_id"].astype("int64")

source["flow_start_timestamp_us"] = source[
    "flow_start_timestamp_us"
].astype("int64")

# 3.
source = source.drop_duplicates(
    "flow_id",
    keep="first",
)

# 4.
source = source.sort_values(
```

```

        ["flow_start_timestamp_us", "flow_id"],
        kind="mergesort",
    ).reset_index(drop=True)

# 5.
with report_path.open(encoding="utf-8") as file:
    report = json.load(file)

all_passed = True

for window in report["windows"]:
    output_path = Path(window["flow_csv"])

    selected = pd.read_csv(
        output_path,
        usecols=["flow_id", "flow_start_timestamp_us"],
    )

    start = window["start_row"]
    count = window["flow_count"]

    #
    expected = source.iloc[
        start:start + count
    ][["flow_id", "flow_start_timestamp_us"]].reset_index(drop=True)

    actual = selected[
        ["flow_id", "flow_start_timestamp_us"]
    ].reset_index(drop=True)

    same_length = len(actual) == len(expected)
    same_flow_ids = actual["flow_id"].equals(
        expected["flow_id"]
    )
    same_timestamps = actual[
        "flow_start_timestamp_us"
    ].equals(
        expected["flow_start_timestamp_us"]
    )

    passed = (
        same_length
        and same_flow_ids
        and same_timestamps
    )

    all_passed = all_passed and passed

    print(
        window["name"],
        "    =", passed,
        "    =", same_length,
        "flow_id    =", same_flow_ids,
        "    =", same_timestamps,
        "    =",
        f"[{start}, {start + count - 1}]",
    )

print("\n    ", all_passed)

```

正确输出应该类似：

```

Wednesday-workingHours-external-w01    = True
Wednesday-workingHours-external-w02    = True
Wednesday-workingHours-external-w03    = True
Wednesday-workingHours-external-w04    = True
Wednesday-workingHours-external-w05    = True

```

True



这能证明每个输出文件确实是：

```
.iloc[start:start + 5000]
```

而不是随机抽取，也不是隔行抽取。

#### ## 四、验证多个窗口是否重叠

```
import pandas as pd
from pathlib import Path

out_dir = Path("dataset/wednesday_external_windows")
flow_files = sorted(out_dir.glob("*-stage1_flows.csv"))

seen = set()

for path in flow_files:
    df = pd.read_csv(path, usecols=["flow_id"])
    current = set(df["flow_id"].astype("int64"))

    overlap = seen & current

    print(
        path.name,
        "    flow  =",
        len(overlap),
    )

    seen.update(current)
```

正确结果应该全部为：

```
flow  = 0
```

---

#### ## 五、验证窗口之间的时间关系

```
import pandas as pd
from pathlib import Path

out_dir = Path("dataset/wednesday_external_windows")
flow_files = sorted(out_dir.glob("*-stage1_flows.csv"))

ranges = []

for path in flow_files:
    df = pd.read_csv(
        path,
        usecols=["flow_start_timestamp_us"],
    )

    start_time = df["flow_start_timestamp_us"].min()
    end_time = df["flow_start_timestamp_us"].max()

    ranges.append((path.name, start_time, end_time))

for i, (name, start, end) in enumerate(ranges):
    print(
        name,
        "start =", start,
        "end =", end,
    )

    if i > 0:
        previous_name, previous_start, previous_end = ranges[i - 1]

        print(
            "    ",
            "
```

```

        "        =",
        start >= previous_end,
    )

```

对于 `--placement even`:

- 每个窗口内部连续。
- 窗口之间通常存在大量未选择的 flow。
- 后一个窗口的开始时间应不早于前一个窗口的结束时间。

对于 `--placement consecutive`:

- 后一个窗口紧跟前一个窗口。
- 可以进一步验证原始行号:

```
current_start == previous_start + previous_count
```

```
---
```

## ## 六、验证 packets 是否属于对应的 flow

Flow 按时间取出后, packet 是通过 `flow\_id` 关联的。可以检查每个 packet 的 `flow\_id` 是否都存在于对应 flow 文件:

```

import pandas as pd
from pathlib import Path

out_dir = Path("dataset/wednesday_external_windows")

for flow_path in sorted(
    out_dir.glob("*-stage1_flows.csv")
):
    packet_path = Path(
        str(flow_path).replace(
            "-stage1_flows.csv",
            "-stage1_packets.csv",
        )
    )

    flow_ids = set(
        pd.read_csv(
            flow_path,
            usecols=["flow_id"],
        )["flow_id"].astype("int64")
    )

    packet_flow_ids = set()

    for chunk in pd.read_csv(
        packet_path,
        usecols=["flow_id"],
        chunksize=500_000,
    ):
        ids = pd.to_numeric(
            chunk["flow_id"],
            errors="coerce",
        ).dropna().astype("int64")

        packet_flow_ids.update(ids.unique())

    unexpected = packet_flow_ids - flow_ids
    missing = flow_ids - packet_flow_ids

    print(
        flow_path.name,
        " packet flow_id =", len(unexpected),
        " packet flow =", len(missing),

```

)

理想结果：

```
packet flow_id = 0
packet flow = 0
```

其中：

- `非法packet flow\_id数 = 0` 是必须满足的。
- `没有packet的flow数 = 0` 是理想情况；如果大于 0，说明原始 flow CSV 中的一些 flow 在原始 packet CSV 里没有对应 packet。

## 最关键的判定标准

建议重点看以下四个结果：

```
= True
= True
flow = 0
packet flow_id = 0
```

四项都满足，就可以确认：

- Flow 是按开始时间排序后连续截取的。
- 不是随机抽取。
- 不同窗口不重复。
- Packet 与选中的 flow 正确关联。

用户 2026-07-30 11:02:45

test: loss=0.325025, acc=0.766000, macro\_precision=0.816012, macro\_recall=0.739925, macro\_f1=0.741264, weighted\_precision=0.804318, weighted\_recall=0.766000, weighted\_f1=0.750640, auc=0.9471190760901217, pr\_auc=0.9115245976467324, precision\_label1=0.9157979149959904, recall\_label1=0.5174444947893068, f1\_label1=0.6612623045744065, cm=[[2688, 105], [1065, 1142]]  
详细解析，FPR

助手 2026-07-30 11:03:31

根据混淆矩阵：

```
cm = [[2688, 105],
      [1065, 1142]]
```

通常 sklearn 的排列方式是：

```
| 真实类别 \ 预测类别 | 预测 benign (0) | 预测 attack (1) |
|---|---|---|
| 真实 benign (0) | TN = 2688 | FP = 105 |
| 真实 attack (1) | FN = 1065 | TP = 1142 |
```

总样本数：

```
2688 + 105 + 1065 + 1142 = 5000
```

## FPR：误报率

FPR (False Positive Rate) 表示：

> 在所有真实 benign 样本中，被模型错误判断为 attack 的比例。

公式：

$$FPR = \frac{FP}{FP + TN}$$

代入：

$$\begin{aligned} FPR &= \frac{105}{105 + 2688} \\ &= \frac{105}{2793} \\ &\approx 0.03759 \end{aligned}$$

因此：

$$FPR = 3.76\%$$

也就是说，在 2,793 个真实 benign flow 中：

- 2,688 个正确识别为 benign。
- 105 个被误报为 attack。
- 每 100 个 benign flow 大约有 3.76 个产生误报。

对应的 specificity（特异度、真负率）为：

$$TNR = 1 - FPR = 96.24\%$$

这个 FPR 相对较低，说明模型对正常流量的误报控制得较好。

## 攻击检测结果

真实攻击样本数：

$$TP + FN = 1142 + 1065 = 2207$$

模型正确检测出：

$$TP = 1142$$

漏掉：

$$FN = 1065$$

### 攻击召回率

$$\begin{aligned} \text{Recall}_1 &= \frac{TP}{TP + FN} \\ &= \frac{1142}{2207} \\ &= 51.74\% \end{aligned}$$

对应输出：

$$\text{recall\_label1} = 0.517444$$

含义是：

> 真实攻击中只有约 51.74% 被检测出来。

因此，虽然 FPR 很低，但漏报情况比较明显。

### FNR：攻击漏报率

$$\begin{aligned} & \backslash[ \\ & \text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} \\ & \backslash] \\ & \backslash[ \\ & \text{FNR} = \frac{1065}{1065 + 1142} \\ & = 48.26\% \\ & \backslash] \end{aligned}$$

即：

$$\text{FNR} = 48.26\%$$

几乎一半的攻击没有被检测到。这是当前结果中最需要注意的问题。

### 攻击精确率

$$\begin{aligned} & \backslash[ \\ & \text{Precision}_1 \\ & = \frac{\text{TP}}{\text{TP} + \text{FP}} \\ & = \frac{1142}{1142 + 105} \\ & = 91.58\% \\ & \backslash] \end{aligned}$$

对应：

$$\text{precision\_label1} = 0.915798$$

含义是：

> 模型一旦报告 attack，大约 91.58% 确实是攻击。

所以当前模型表现为：

- 报警比较谨慎。
- 报警可信度较高。
- 误报较少。
- 但漏掉了大量攻击。

### 攻击 F1

$$\begin{aligned} & \backslash[ \\ & \text{F1}_1 \\ & = \\ & 2 \times \\ & \frac{\text{Precision}_1 \times \text{Recall}_1}{\text{Precision}_1 + \text{Recall}_1} \\ & \backslash] \end{aligned}$$

结果：

$$\text{f1\_label1} = 66.13\%$$

由于攻击 precision 很高但 recall 偏低，最终 F1 被 recall 明显拉低。

## 正常类别表现

真实 benign 数量：

$$TN + FP = 2688 + 105 = 2793$$

benign recall，也就是 specificity：

$$\begin{aligned} \text{Recall}_0 &= \frac{TN}{TN+FP} \\ &= \frac{2688}{2793} \\ &= 96.24\% \end{aligned}$$

benign precision：

$$\begin{aligned} \text{Precision}_0 &= \frac{TN}{TN+FN} \\ &= \frac{2688}{2688+1065} \\ &\approx 71.62\% \end{aligned}$$

这里 benign precision 不高，是因为 1,065 个攻击被错误预测成了 benign。

## Accuracy

$$\begin{aligned} \text{Accuracy} &= \frac{TP+TN}{\text{Total}} \end{aligned}$$

$$\begin{aligned} \text{Accuracy} &= \frac{1142+2688}{5000} \\ &= \frac{3830}{5000} \\ &= 76.60\% \end{aligned}$$

与输出一致：

$$\text{acc} = 0.766$$

错误样本共：

$$105 + 1065 = 1170$$

其中大多数错误来自攻击漏报：

$$\begin{aligned} \text{FN} &= 1065 / 1170 \\ &\approx 91.03\% \end{aligned}$$

也就是说，模型错误中约 91% 是漏报，而不是误报。

## Macro Recall

两类 recall 分别为：

```
Recall_0 = 96.24%
Recall_1 = 51.74%
```

Macro Recall：

```
\[
\frac{96.24\%+51.74\%}{2}
\approx 73.99\%
\]
```

对应：

```
macro_recall = 0.739925
```

在二分类任务中，Macro Recall 也等于 balanced accuracy。它比普通 accuracy 更能反映两类是否都检测得好。

## Macro Precision

两类 precision：

```
Precision_0 = 71.62%
Precision_1 = 91.58%
```

平均：

```
macro_precision = 81.60%
```

说明模型预测结果总体比较可信，但这不能反映模型漏掉多少攻击，因此必须结合 macro recall 阅读。

## Macro F1 与 Weighted F1

```
macro_f1 = 74.13%
weighted_f1 = 75.06%
```

Macro F1 对 benign 和 attack 两类给予相同权重，更适合分析两类是否均衡。

Weighted F1 按类别样本数量加权。当前数据中：

```
benign = 2793    55.86%
attack  = 2207    44.14%
```

类别本身不算严重失衡，因此 macro F1 和 weighted F1 差距不大。

## ROC-AUC

```
AUC = 0.9471
```

这是一个很高的 ROC-AUC，说明从预测概率排序角度看，模型通常能给攻击样本比 benign 样本更高的风险分数。

但 AUC 高、当前 attack recall 只有 51.74%，并不矛盾：

- AUC 衡量所有可能阈值下的总体排序能力。
- 混淆矩阵只对应当前的一个分类阈值。
- 当前阈值可能偏保守，导致 FPR 很低，但攻击漏报很多。

因此模型可能具备良好的区分能力，只是当前决策阈值未取得合适的 FPR/recall 平衡。

## PR-AUC

```
PR-AUC = 0.9115
```

PR-AUC 同时关注攻击类别的 precision 和 recall。0.9115 表明概率输出对攻击类别具有较好的整体排序能力。

当前具体阈值下：

```
Precision_1 = 91.58%  
Recall_1    = 51.74%
```

说明该阈值选择了 PR 曲线中“高 precision、低 recall”的区域。

## Loss

```
loss = 0.325025
```

如果使用的是二分类交叉熵，loss 不仅与预测是否正确有关，还与预测概率的置信程度有关。

它不能像 accuracy 一样单独直观解释，主要用于：

- 比较不同 epoch。
- 比较验证集和测试集差距。
- 判断是否过拟合。
- 比较相同损失定义下的不同模型。

单看 `0.325` 无法判断优劣，需要和训练 loss、validation loss 以及其他模型的 test loss 对照。

## 综合评价

当前模型的主要特点是：

| 指标               | 结果     | 含义            |
|------------------|--------|---------------|
| Accuracy         | 76.60% | 总体正确率一般       |
| FPR              | 3.76%  | 正常流量误报较低      |
| Specificity      | 96.24% | 正常流量识别很好      |
| Attack precision | 91.58% | 报警可信度高        |
| Attack recall    | 51.74% | 只检测出约一半攻击     |
| FNR              | 48.26% | 攻击漏报较严重       |
| Attack F1        | 66.13% | 被较低 recall 拉低 |
| ROC-AUC          | 94.71% | 概率排序能力很好      |
| PR-AUC           | 91.15% | 攻击排序能力较好      |

结论是：

> 模型不是缺少区分攻击与正常流量的能力，而是当前工作点明显偏保守：以较低的 3.76% FPR 换取了较高的 48.26% FNR。

如果 IDS 更重视“不能漏掉攻击”，可以适当降低分类阈值，提高 attack recall；代价是 FPR 会升高。阈值应当只在 validation set 上选择，例如寻找：

```
Recall_label1 ≥ 80%
```

时 FPR 最低的阈值，或者寻找：

```
FPR ≤ 5%
```

约束下 attack recall 最高的阈值。确定阈值后再固定应用到这个外部 test set，不能直接用 test set 调阈值，否则会产生测试集信息泄露。